

ÖRNEK 1)

Öğrenci Kayıt Sistemi

Numaraya göre sıralı öğrenci ekleyen, listeyi hem baştan hem sondan yazdıran çift bağlı liste uygulamasını c koduyla yazınız.

İstenilenler:

- Node yapısı: numara, isim, next, prev
- insertOrdered() → numaraya göre sıralı ekleme
- displayForward() → baştan sona yazdır
- displayBackward() → sondan başa yazdır

ÖDEV 1)

Bu uygulama bir müzik çalar gibi çalışacak.

Kullanıcı şarkı ekleyip silebilecek, ileri/geri şarkı geçişi yapabilecek.

Buradaki amaç, **prev** ve **next** pointer'larının dinamik şekilde yönetimini kavratmak.

Yapılacaklar:

- addSongToEnd(Node** head, char* isim)
- removeSong(Node** head, char* isim)
- playNext() ve playPrevious() fonksiyonları.
- Menü: Kullanıcı seçimleriyle listeyi yönetsin.

İpucu:

- Şarkı silerken prev ve next bağlantılarını düzgün güncelleyin.
- Liste boşken "Liste boş" uyarısı verin.

Kod iskeleti:

```
typedef struct Song {  
    char name[50];  
    struct Song* next;  
    struct Song* prev;  
} Song;
```

// Fonksiyon imzaları:

void addSongToEnd(Song** head, char* name);

void removeSong(Song** head, char* name);

void playNext(Song** current);

void playPrevious(Song** current);

void displayPlaylist(Song* head);

ÖRNEK 2)

Parantez Dengesi Kontrolü

İstenilenler:

- Kullanıcıdan bir ifade alınır (örnek: {[()()]})
- Stack kullanarak parantezlerin dengede olup olmadığı kontrol edilir.
- Her (için push, her) için pop yapılır.

Beklenen Çıktı:

- İfade: {[()()]}
- Sonuç: Parantezler dengede!

ÖDEV 2)

Undo (Geri Alma) Özelliği Simülasyonu

İstenilenler:

- Kullanıcı “metin ekle” diyerek bir kelime girer → stack’e eklenir.
- “undo” seçerse son eklenen kelime geri alınır (pop edilir).
- “show” seçerse şu ana kadar eklenen kelimeleri gösterir.

Beklenen örnek kullanım:

- > add Merhaba
- > add Dünya
- > show -> Merhaba Dünya
- > undo
- > show -> Merhaba

Kod iskeleti:

```
typedef struct Word {
    char text[50];
    struct Word* next;
} Word;

void pushWord(Word** top, char* text);
void popWord(Word** top);
void showWords(Word* top);
```

ÖRNEK 3)

İstenilenler:

- Node: müşteriNumarası, next
- Fonksiyonlar:
 - enqueue(), dequeue(), display()

Kullanıcı “müşteri geldi” dediğinde kuyruğa eklenir.
“işlem tamamlandı” dediğinde sıradaki müşteri çıkar.
display() ile bekleyen müşteri sırası gösterilir.

Beklenen çıktı:

Müşteri #1 geldi.
Müşteri #2 geldi.
Sıradaki müşteri: #1
Müşteri #1 işlemi tamamladı.
Yeni sıradaki müşteri: #2

ÖDEV 3)

Bu uygulama bir yazıcının işlem sırasını taklit edecek.
Her eklenen dosya kuyruğa alınır, “Yazdır” komutuyla sıradaki iş çıkarılır.

Yapılacaklar:

- enqueuePrintJob(Queue* q, char* dosyaAdı)
- processNextJob(Queue* q)
- showQueue(Queue q)
- Menü: 1) Yeni dosya ekle, 2) Yazdır, 3) Kuyruğu göster

İpucu:

- Kuyruk boşken yazdırma işlemi yapılmamalı.

- FIFO mantığına dikkat edin.

Kod iskeleti:

```
typedef struct PrintJob {  
    char fileName[50];  
    struct PrintJob* next;  
} PrintJob;  
  
typedef struct Queue {  
    PrintJob* front;  
    PrintJob* rear;  
} Queue;  
  
void enqueuePrintJob(Queue* q, char* fileName);  
void processNextJob(Queue* q);  
void showQueue(Queue q);
```